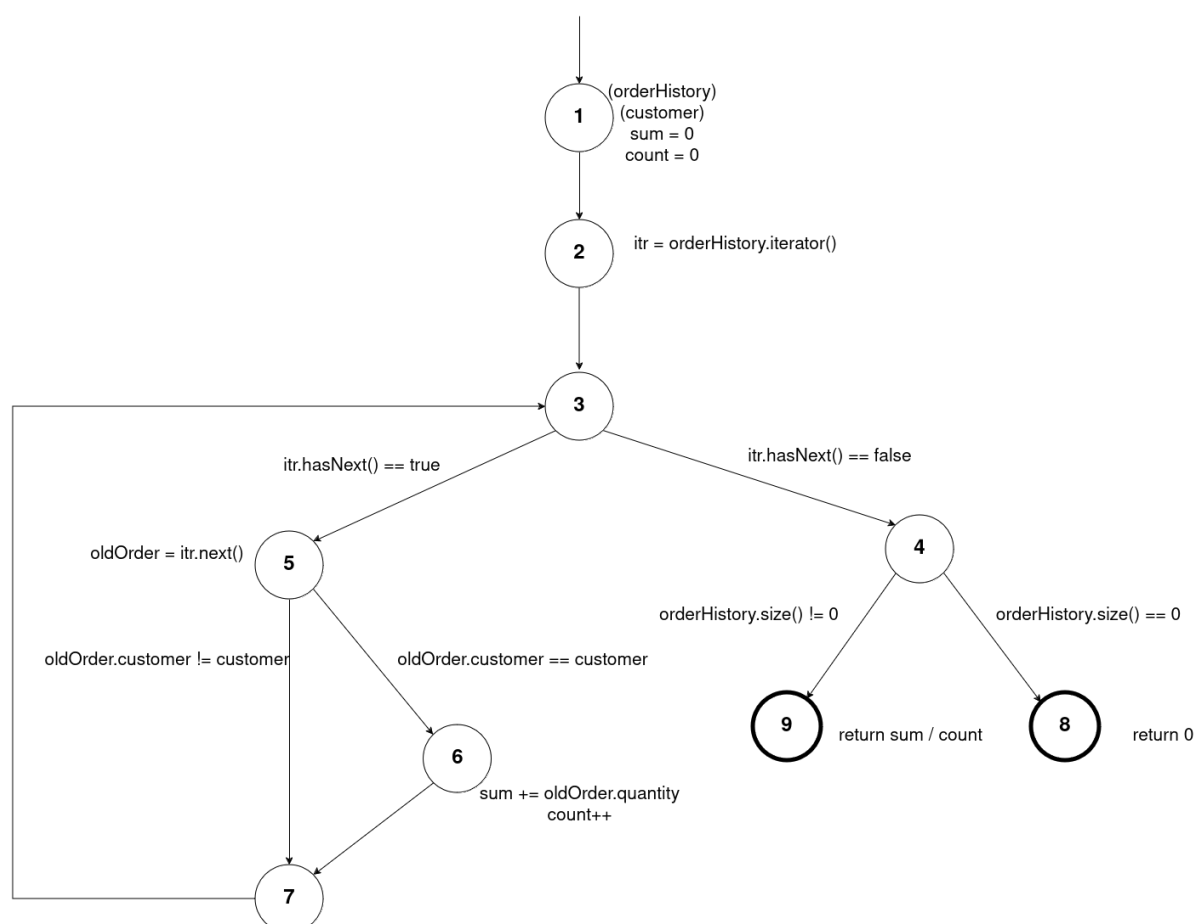


سوال اول

(الف)

CFG:



Prime Paths:

[1, 2, 3, 4, 8]! - [1, 2, 3, 4, 9]! - [1, 2, 3, 5, 6, 7] - [1, 2, 3, 5, 7] - [3, 5, 6, 7, 3]* - [3, 5, 7, 3]* -
 [5, 6, 7, 3, 5]* - [5, 7, 3, 5]* - [6, 7, 3, 5, 6]* - [7, 3, 5, 7]* - [7, 3, 5, 6, 7]* - [5, 6, 7, 3, 4, 8]! -
 [5, 6, 7, 3, 4, 9]! - [5, 7, 3, 4, 8]! - [5, 7, 3, 4, 9]!

DU Paths:

Variable	DU Pairs	DU Paths
orderHistory	(1, 2) (1, (4, 8)) (1, (4, 9))	1. [1, 2] 2. [1, 2, 3, 4, 8]! 3. [1, 2, 3, 4, 9]!
customer	(1, (5, 6)) (1, (5, 7))	4. [1, 2, 3, 5, 6] 5. [1, 2, 3, 5, 7]
sum	(1, 9) (1, 6) (6, 6) (6, 9)	3. [1, 2, 3, 4, 9]! 4. [1, 2, 3, 5, 6] 6. [6, 7, 3, 5, 6]* 7. [6, 7, 3, 4, 9]!
count	(1, 9) (1, 6) (6, 6) (6, 9)	3. [1, 2, 3, 4, 9]! 4. [1, 2, 3, 5, 6] 6. [6, 7, 3, 5, 6]* 7. [6, 7, 3, 4, 9]!
itr	(2, (3, 4)) (2, (3, 5)) (2, 5)	8. [2, 3, 4] 9. [2, 3, 5] 9. [2, 3, 5]
oldOrder	(5, (5, 7)) (5, (5, 6)) (5, 6)	10. [5, 7] 11. [5, 6] 11. [5, 6]

ب) Prime Path-ها به همان ترتیبی که نوشته شده‌اند در نظر بگیرید و با ایندکس آن‌ها را نشان می‌دهیم.

Test-Path	TRs Toured	
	Prime Path Index	DU Path Index
[1, 2, 3, 4, 8]	1	1, 2, 8
[1, 2, 3, 5, 6, 7, 3, 5, 6, 7, 3, 4, 9]	3, 5, 7, 9, 11, 13	1, 4, 6, 7, 9, 11
[1, 2, 3, 5, 7, 3, 5, 7, 3, 4, 9]	4, 6, 8, 10, 15	1, 5, 9, 10

مورد ۲، ۱۲، ۱۴ از Prim Path-ها و مورد ۳ از DU Path-ها Infeasible هستند زیرا اگر وارد حلقه شویم یعنی لیست خالی نیست در نتیجه وارد If بعد از آن نمی‌شویم و اگر وارد حلقه نشویم یعنی لیست خالی است پس قطعاً وارد If بعد از آن می‌شویم.

ج) تابع مقدار کف میانگین را بر می‌گرداند و ما فرض می‌کنیم که این درست و خطایی در تایپ خروجی و sum و count نیست.

خطای اصلی این است که وقتی لیست ما خالی نباشد و مقدار customer هیچ عنصری از آن با مقدار customer ورودی تابع برابر نباشد. ArithmeticException از نوع Division by zero رخ می‌دهد در حالی که ما انتظار داریم در این حالت مقدار صفر را برگرداند. این خطا در تست سوم دیده می‌شود. و برای رفع آن می‌توانیم این بخش کد را:

```
if (orderHistory.size() == 0){
    return 0;
}
```

به این صورت شرط آن را تغییر دهیم:

```
if (count == 0){
    return 0;
}
```

سوال دوم

الف) یک مثال از آن می‌تواند به شکل زیر باشد.

- پوشش جمله صد درصدی: abc = TTF, FFT
- پوشش شاخه صد درصدی: abc = TTF, FFT, FFF

اگر بخواهیم هر دو را داشته باشیم کافیست پوشش شاخه صد درصدی را برای ساخت آزمایش در نظر بگیریم.

ب) یک راه آن می‌تواند حذف شرط ها باشد:

```
var1 = a | b;
var2 = !a & !b & c;

return var1, var2;
```

یا اگر بخواهیم به صورت if-statement بنویسیم، می‌توانیم به صورت زیر بازنویسی کنیم:

```

var1, var2 = false, true;
if (a | b) {
    var1 = true;
}
if (a | b | !c) {
    var2 = false;
}
return var1, var2;

```

برای کد اول به ازای هر ورودی و برای کد دوم به طور مثال به ازای $abc = TTF$ پوشش جمله صد درصدی بدست می‌آید.

ج) فرض کنید CFG ما n نود و e یال داشته باشد. یک شرط کافی برای اینکه بتوان با یک آزمایش به پوشش جمله صد درصدی رسید این است که $e = n - 1$ باشد و گراف یک مسیر جهت دار به طول $(n - 1)$ داشته باشد.

دقت کنید که اگر در گراف حلقه وجود داشته باشد که ناشی از شرط یا حلقه در کد است دیگر نمی‌توانیم اطمینان دهیم. زیرا یا ممکن است شرط‌ها نتوانند همزمان True شوند یا حتی اگر فقط یک شرط هم داشته باشیم ممکن است آن شرط همیشه False باشد. در صورتی که چنین چیزی از روی گراف نمی‌توان تشخیص داد.

سوال سوم

برای آن مثال نقض می‌آوریم. یک تابع که شامل هیچ متغیری نیست را در نظر بگیرید:

```

public void printSoftwareTestingCourse() {
    System.out.println("Software testing course");
}

```

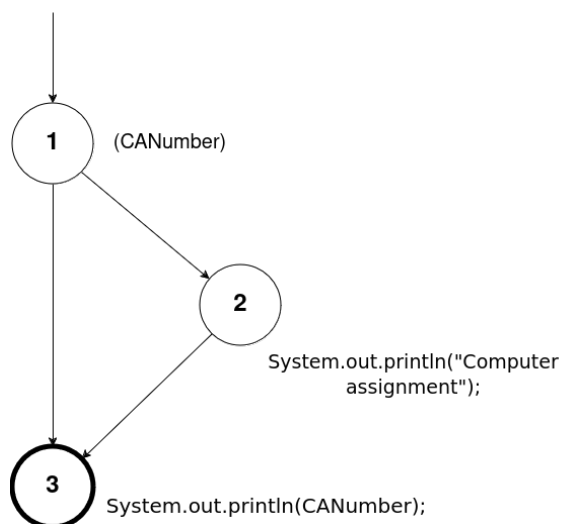
این تابع هیچ DU path-ای ندارد و برای پوشش کامل مسیرهای DU نیاز به هیچ تستی ندارد. در حالی که یک Prime path به طول صفر دارد (یک نود تنها که شامل تنها جمله تابع است) که حداقل نیاز به یک تست دارد تا پوشش داده شود. اگر بخواهیم مثالی بیاوریم که شامل DU نیز باشد:

```

public void printComputerAssignmentNumber(int CANumber) {
    if (true) {
        System.out.println("Computer assignment:");
    }
    System.out.println(CANumber);
}

```

CFG این کد به شکل زیر می باشد:



که می‌توان با آزمایش [1, 3] به ADUPC رسید در حالی که دو Prime path دارد که یکی از آنها پوشش داده نشده است یعنی مسیر [1, 2, 3] با آزمایش گفته شده پوشش داده نمی‌شود.